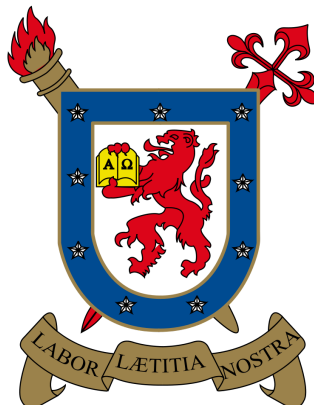




UNIVERSIDAD DE SANTIAGO DE CHILE

FACULTAD DE INGENIERÍA

Departamento de Ingeniería Informática

Optimización de Turnos y Disponibilidad de Gig Workers en Tiempo Real

Innovación y emprendimiento: Desafío 2

Integrantes:

- Stephan Paul.
- Victor Duarte.
- Milovan Valenzuela.
- Benjamin Moya.
- Alonso Henriquez.

Docente:

Francisco Parra

1. Índice

1. Índice	1
2. Resumen Ejecutivo	2
3. Análisis del problema	3
3.1. Descripción detallada del problema a resolver	3
3.2. Impacto en el sector de gig workers	3
3.3. Análisis de soluciones existentes y sus limitaciones	4
4. Propuesta de solución	7
4.1. Arquitectura técnica	7
4.2. Flujos de interacción vía WhatsApp	8
4.3. Modelos implementados	9
4.4. Diagrama de la solución	10
5. Estado actual de desarrollo	11
5.1. Funcionalidades implementadas (con capturas de pantalla)	11
5.1.1 Modelo logístico	11
5.1.2 Cálculo de probabilidades	11
5.1.3 Modelo logístico autorregresivo	12
5.1.4 Modelo SARIMAX	13
5.2. Integración con WhatsApp	13
5.3. Resultados de pruebas preliminares	13
5.4. Porcentaje de avance por componente	16
6. Desafíos y aprendizajes	17
6.1. Obstáculos técnicos encontrados	17
6.2. Adaptaciones a la propuesta inicial.	18
6.3. Lecciones aprendidas	18
7. Plan de trabajo restante	19
7.1. Cronograma detallado de actividades pendientes	19
7.2. Distribución de tareas entre integrantes	19
7.3. Estimación de esfuerzo requerido	19
8. Anexos	22
8.1. Código relevante (extractos):	22
8.2. Documentación técnica adicional	23

2. Resumen Ejecutivo

El problema actual es la falta de herramientas eficientes que permitan a las empresas de delivery gestionar la disponibilidad de sus trabajadores en tiempo real, lo que genera desorganización y pérdida de tiempo. En la actualidad, las soluciones existentes dependen de sistemas de baja precisión, que no logran proporcionar la exactitud necesaria para una gestión óptima de los turnos.

El proyecto tiene como objetivo diseñar un sistema eficiente para gestionar la disponibilidad de los trabajadores en plataformas de delivery y logística. Este sistema se enfocará en que los trabajadores puedan indicar, a través de WhatsApp, los horarios en los que están disponibles para recibir tareas en tiempo real, mejorando así la precisión en la asignación de turnos. Además, el sistema integrará un modelo predictivo para anticipar las ausencias o faltas de los trabajadores, basándose en el análisis de patrones históricos de comportamiento.

El desafío consiste en desarrollar una solución que opere completamente dentro de WhatsApp. El sistema incluirá respuestas automáticas y notificaciones inteligentes, garantizando una operación ágil y sin interrupciones. Las restricciones del proyecto estipulan que la plataforma debe ser 100% funcional en WhatsApp, sin integrar aplicaciones adicionales.

3. Análisis del problema

3.1. Descripción detallada del problema a resolver

En el ámbito de las empresas de delivery y logística, la gestión de la disponibilidad de los trabajadores es un desafío crítico para garantizar una operación eficiente y sin descoordinaciones. Actualmente, las empresas enfrentan dificultades en la asignación de turnos o tareas a sus gig workers, debido a que los sistemas utilizados para gestionar su disponibilidad son ineficaces o de baja precisión.

Uno de los principales problemas consiste en que los trabajadores tienen que informar su disponibilidad mediante llamadas telefónicas o sistemas de baja precisión, lo que no solo genera una gran cantidad de trabajo administrativo, sino que también da lugar a errores de asignación. Este tipo de sistemas no garantiza una actualización en tiempo real ni una coordinación eficiente, lo que puede resultar en pérdidas de tiempo, desorganización y, en última instancia, una mala experiencia tanto para los trabajadores como para los clientes que esperan sus entregas.

Por otro lado, las empresas también enfrentan el problema de predecir las ausencias de los trabajadores o las faltas de disponibilidad, lo cual es fundamental para optimizar la asignación de turnos. La falta de herramientas predictivas para identificar patrones de comportamiento de los trabajadores (como la tendencia a faltar o a no cumplir con su disponibilidad previamente acordada) hace que las empresas se vean obligadas a lidiar con cambios de última hora, lo que afecta la eficiencia operativa.

El problema central radica en desarrollar un sistema integrado y preciso que permita gestionar de manera eficiente la disponibilidad de los trabajadores en tiempo real, predecir posibles ausencias y evitar los errores en la asignación de tareas para reducir la desorganización.

3.2. Impacto en el sector de gig workers

El impacto de un sistema ineficaz de gestión de disponibilidad y asignación de turnos en el personal de alta rotación, como los gig workers, es considerable tanto a nivel individual como organizacional. Este tipo de trabajadores, que no tienen un contrato fijo y a menudo trabajan en múltiples plataformas o para diferentes empresas, requieren una gestión flexible y eficiente que se adapte a sus necesidades y horarios.

1. **Incertidumbre en la asignación de tareas:** Los gig workers dependen de la posibilidad de recibir tareas a lo largo del día. Cuando el sistema de gestión es impreciso o lento, esto genera incertidumbre sobre si podrán trabajar en el momento que desean o si se les podrán asignar tareas a tiempo. La falta de información en tiempo real aumenta la frustración y la desmotivación de los trabajadores, lo que podría llevar a un menor rendimiento y, en algunos casos, a la pérdida de

trabajadores clave.

2. **Desorganización y falta de flexibilidad:** Los trabajadores de alta rotación valoran la flexibilidad en sus horarios. Si la asignación de turnos no está optimizada o se basa en sistemas anticuados, puede haber choques o falta de coincidencia entre la disponibilidad del trabajador y las tareas asignadas. Esto no solo afecta la satisfacción del trabajador, sino que también puede llevar a la sobrecarga de trabajo en algunos casos y la falta de trabajo en otros.
3. **Bajas expectativas y desconfianza:** Cuando los gig workers no tienen certeza sobre la disponibilidad de trabajo, esto puede generar desconfianza en la plataforma o la empresa que los emplea. Esta falta de previsibilidad puede hacer que busquen otras plataformas que ofrezcan una mayor estabilidad o claridad, lo que afecta la retención y genera altos índices de rotación de personal.
4. **Impacto en las ganancias del trabajador:** Un sistema poco eficiente puede llevar a que los gig workers pierdan oportunidades de trabajo. Si los turnos no son asignados de manera oportuna o precisa, los trabajadores podrían no recibir suficientes tareas para generar un ingreso adecuado, lo que afecta negativamente sus ganancias y su motivación para continuar trabajando con la plataforma o empresa.
5. **Falta de apoyo ante ausencias imprevistas:** Los gig workers a menudo tienen dificultades para informar sus ausencias o cambios en la disponibilidad de manera eficiente. Sin una herramienta adecuada para gestionar estos imprevistos, las empresas pueden asignar tareas a trabajadores que no están disponibles, lo que genera más frustración y estrés para el trabajador, además de afectar la eficiencia de la operación.

En conclusión, la falta de un sistema eficaz para gestionar la disponibilidad de los gig workers tiene un impacto directo en la satisfacción, retención y desempeño de estos trabajadores. Un sistema que permita la asignación precisa y en tiempo real, así como la predicción de ausencias, puede mejorar la experiencia del trabajador, reduciendo la rotación y mejorando las ganancias y la estabilidad de la fuerza laboral.

3.3. Análisis de soluciones existentes y sus limitaciones

Las soluciones para gestionar a los trabajadores se pueden agrupar en dos grandes categorías: plataformas y software tecnológicos, y estrategias de gestión y recursos humanos.

Soluciones Actuales para la Gestión de Gig Workers

1. Plataformas y Software Tecnológicos

La tecnología es un pilar fundamental en la gestión de la fuerza laboral donde destacan:

- **Software de Gestión de Recursos Humanos (HRM):** Sistemas como Sage HR, BambooHR, Bizneo HR Suite, Factorial HR, Personio y Deel están adaptándose o pueden configurarse para administrar ciertos aspectos de los *gig workers*. Facilitan la centralización de datos, la automatización de procesos de *onboarding* y *offboarding*, y en algunos casos, la gestión de pagos.
- **Plataformas de Gestión de Freelancers y Talento Externo:** Estas plataformas (a menudo llamadas *Freelancer Management Systems* - FMS) conectan a las empresas con trabajadores independientes y ayudan a gestionar todo el ciclo de vida de la colaboración: desde la búsqueda y selección (ej. LinkedIn, plataformas especializadas como Seeds) hasta la contratación, gestión de proyectos y pagos.
- **Soluciones Fintech:** Las tecnologías financieras ofrecen herramientas para pagos flexibles y ágiles (billeteras electrónicas, transferencias rápidas), a menudo con comisiones más bajas, lo cual es crucial para los *gig workers*. Algunas incluso exploran opciones de microcréditos o adelantos de facturas.
- **Herramientas de Gestión de Proyectos y Colaboración:** Aplicaciones como Asana, Trello, Slack, Microsoft Teams, o ProofHub son esenciales para asignar tareas, dar seguimiento a los proyectos, compartir documentos y mantener una comunicación fluida entre los equipos internos y los *gig workers*.

2. Estrategias de Gestión y Recursos Humanos

Más allá de la tecnología, las prácticas de gestión son cruciales:

- **Procesos de Onboarding Ágiles y Específicos:** Facilitar una rápida integración del *gig worker* al proyecto y a la cultura de la empresa, definiendo claramente expectativas, entregables y canales de comunicación.
- **Comunicación Clara y Constante:** Establecer canales de comunicación efectivos y proporcionar retroalimentación regular sobre el desempeño.
- **Compensación Justa y Transparente:** Ofrecer tarifas competitivas y asegurar pagos puntuales.

Limitaciones de las Soluciones Actuales

A pesar de los avances, la gestión de *gig workers* y las soluciones existentes enfrentan importantes desafíos y limitaciones:

1. Limitaciones Inherentes al Modelo Gig y Desafíos para los Trabajadores

Muchas limitaciones no provienen de las soluciones en sí, sino del propio modelo de trabajo *gig*:

- **Falta de Beneficios y Protección Social:** La mayoría de los *gig workers* no acceden a beneficios laborales tradicionales como seguro médico, vacaciones pagadas, cotizaciones para la jubilación o seguro de desempleo. Las plataformas actuales de gestión raramente solucionan este vacío estructural.

- **Inestabilidad Laboral y de Ingresos:** La naturaleza proyectual del trabajo gig implica una falta de seguridad laboral a largo plazo y una potencial fluctuación en los ingresos, dificultando la planificación financiera.

2. Limitaciones para las Empresas y la Gestión

Las empresas también enfrentan obstáculos al implementar estas soluciones:

- **Complejidad Legal y Regulatoria:**
 - **Clasificación de Trabajadores:** Distinguir entre un contratista independiente y un empleado es un desafío legal crucial, con implicaciones fiscales y de cumplimiento laboral significativas. Un error en la clasificación puede acarrear sanciones. Las plataformas pueden ayudar a gestionar contratos, pero la responsabilidad final de la clasificación recae en la empresa.
- **Seguridad de Datos y Propiedad Intelectual:** Gestionar el acceso a información sensible y asegurar la protección de la propiedad intelectual con trabajadores externos requiere políticas y tecnologías robustas.
- **Garantía de Calidad y Consistencia:** Asegurar que el trabajo de los *gig workers* cumpla consistentemente con los estándares de calidad de la empresa puede requerir un esfuerzo de supervisión y feedback más intensivo.
- **Dependencia Tecnológica y Costos:** Si bien la tecnología es una solución, también puede generar dependencia y los costos asociados a múltiples plataformas los cuales pueden ser significativos.

En conclusión, si bien existen numerosas soluciones que facilitan enormemente la incorporación y gestión de *gig workers*, es fundamental que las empresas sean conscientes de sus limitaciones. Una gestión exitosa requiere una combinación de tecnología adecuada, estrategias de recursos humanos bien pensadas y un entendimiento claro del marco legal y las necesidades tanto de la organización como de los trabajadores independientes.

4. Propuesta de solución

La solución propuesta se centra en desarrollar un sistema integral y eficiente, operando completamente a través de WhatsApp, para optimizar la gestión de turnos y la disponibilidad de los gig workers. Este sistema se estructura en tres componentes modulares principales: indicación de horarios de actividad, predicción de faltas basada en patrones de comportamiento, y asignación y recepción de tareas en tiempo real.

4.1. Arquitectura técnica

La arquitectura técnica de la solución se concibe como un sistema multicapa, diseñado para ser robusto, escalable y operar en tiempo real, con WhatsApp como la interfaz principal para el gig worker.

Interfaz de Usuario (Frontend):

- **WhatsApp:** La interacción principal con los gig workers se realizará exclusivamente a través de la API de WhatsApp Business. Esto permite a los trabajadores comunicar su disponibilidad, recibir asignaciones de tareas y otras notificaciones directamente en una aplicación que ya utilizan ampliamente.

Capa de Aplicación (Backend):

- **Servidor de Aplicaciones:** Un servidor central que gestionará la lógica de negocio.
- **API Gateway para WhatsApp:** Este componente recibirá los mensajes entrantes desde la API de WhatsApp, los procesará y dirigirá las solicitudes a los módulos correspondientes. También formateará las respuestas para enviarlas de vuelta al trabajador vía WhatsApp.
- **Módulo de Gestión de Disponibilidad:** Encargado de procesar las indicaciones de horarios de los trabajadores, actualizando su estado en la base de datos.
- **Módulo de Asignación de Tareas:** Contendrá los algoritmos para la asignación de tareas en tiempo real, considerando la disponibilidad, ubicación, y otros criterios relevantes.
- **Módulo de Notificaciones:** Gestionará el envío de notificaciones automáticas e inteligentes (confirmaciones, recordatorios, nuevas tareas, etc.) a los trabajadores.

Capa de Datos:

- **Base de Datos:** Se utilizará una base de datos para almacenar información persistente como:
 - Perfiles de los gig workers.
 - Historial de disponibilidad y turnos trabajados.
 - Datos históricos de comportamiento (para el modelo predictivo).

- Registro de tareas y su estado.

Módulo de Inteligencia Artificial (Machine Learning):

- **Servicio de Modelos Predictivos:** Un componente separado o integrado en el backend que alojará y ejecutará los modelos de machine learning para predecir ausencias. Este servicio recibirá datos históricos y en tiempo real para generar predicciones.

Integraciones:

- **API de WhatsApp Business:** Esencial para la comunicación bidireccional.
- **Posibles APIs Internas de Fletzy:** Para sincronizar información de tareas, trabajadores, o cualquier otro sistema relevante que Fletzy ya posea (como el sistema existente para indicar horarios, al cual se adaptará o complementará).

4.2. Flujos de interacción vía WhatsApp

Aunque el desarrollo de la integración con WhatsApp aún no se ha iniciado, se ha definido que este será el canal principal de comunicación e interacción entre el sistema y los gig workers. La propuesta de flujo de trabajo a través de WhatsApp se centra en dos procesos clave: la confirmación de turnos y la asignación de tareas.

1. Confirmación de Turnos a través de WhatsApp:

- **Objetivo:** Verificar la disponibilidad de los trabajadores para el día siguiente y conocer su hora de inicio preferida.
- **Flujo:**
 - El sistema enviará proactivamente un mensaje a los trabajadores consultando su disponibilidad para la operación programada para el día siguiente.
 - En este mensaje, se les solicitará que confirmen su asistencia y que especifique la hora a la que podrían comenzar a trabajar.

Ejemplo de mensaje enviado por el sistema:

"Hola, ¿Confirma su asistencia a la operación programada para mañana? Confirma tu disponibilidad y la hora de inicio."

- **Respuesta del trabajador:** El trabajador deberá responder indicando su confirmación y la hora de inicio (ej. "Sí, puedo desde las 09:00", "Confirmo, 10:00 AM").
- **Acción del sistema:** El sistema registrará las respuestas, actualizando la disponibilidad y hora de inicio de los trabajadores que confirmen.

2. Asignación de Tareas mediante WhatsApp:

- **Objetivo:** Asignar tareas específicas a los trabajadores que han confirmado su disponibilidad, utilizando un modelo de "primer llegado, primer servido".

- **Flujo:**
 - Cuando una nueva tarea esté disponible, el sistema enviará un mensaje de notificación a **todos** los trabajadores que hayan confirmado previamente su disponibilidad para ese día.
 - El mensaje informará sobre la disponibilidad de una tarea y solicitará una respuesta afirmativa para tomarla.
 - El primer trabajador que responda afirmativamente al mensaje será a quien se le asigne la tarea.

Ejemplo de mensaje enviado por el sistema:

"Hay una tarea disponible. Si estás disponible para tomarla, responde 'Sí' y te asignaremos la tarea."

- **Respuesta del trabajador:** El trabajador interesado responderá "Sí" (u otra palabra clave afirmativa definida).
- **Acción del sistema:**
 - Al recibir la primera respuesta afirmativa, el sistema asignará la tarea a ese trabajador y podría enviar una confirmación adicional.
 - A los trabajadores que respondan afirmativamente después del primero, se les podría notificar que la tarea ya fue asignada.

Estos flujos buscan simplificar la comunicación, agilizar la confirmación de disponibilidad y asegurar un método rápido y transparente para la asignación de tareas, aprovechando la inmediatez y familiaridad de WhatsApp para los gig workers.

4.3. Modelos implementados

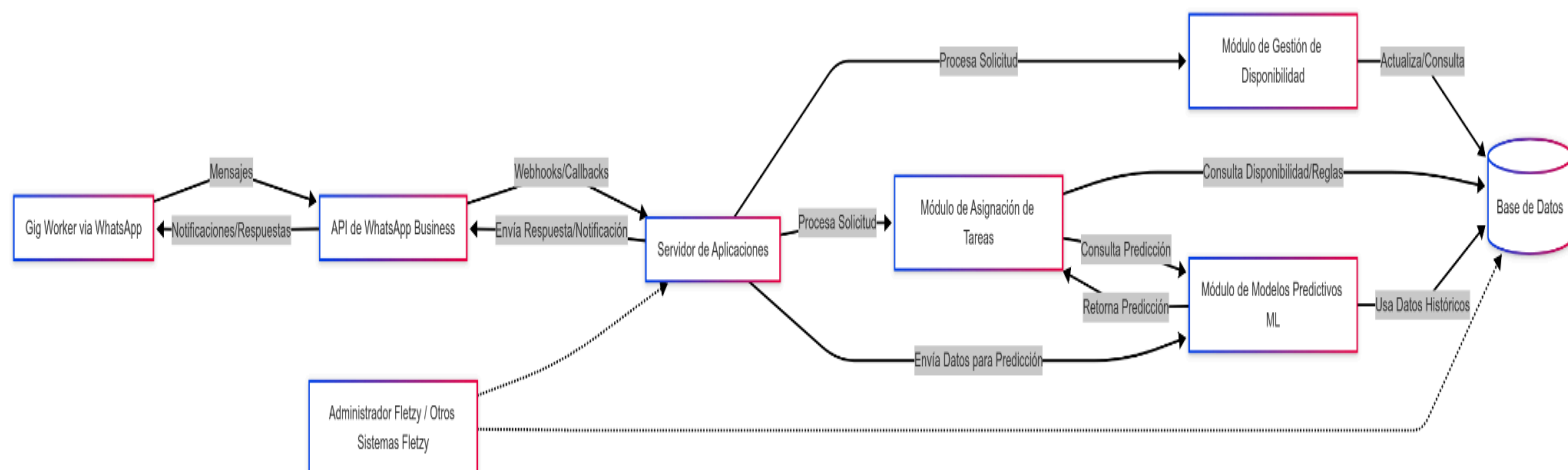
El componente de predicción de faltas se basa en el análisis de patrones históricos de comportamiento de los trabajadores. Los modelos implementados y en fase de prueba y optimización son:

1. **SARIMAX (Seasonal AutoRegressive Integrated Moving Average with exogenous factors):**
 - **Propósito:** Modelar y predecir series temporales que pueden tener componentes de estacionalidad (ej. patrones semanales o mensuales de disponibilidad/ausencia) y pueden ser influenciadas por variables externas (exógenas).
 - **Datos de Entrada (potenciales):** Historial de asistencias/ausencias, día de la semana, hora del día, tipo de turno, historial previo del trabajador, posiblemente datos externos como eventos especiales o clima (si se dispone de ellos).
 - **Salida:** Predicción de la probabilidad de ausencia para un turno futuro.
2. **Autoregresivo (AR):**

- **Propósito:** Un modelo más simple que predice valores futuros basándose en una combinación lineal de valores pasados de la misma variable (en este caso, la asistencia o ausencia).
- **Datos de Entrada:** Historial de asistencias/ausencias del trabajador.
- **Salida:** Predicción de la probabilidad de ausencia.

Proceso de Implementación de Modelos:

- **Generalización:** Actualmente, los modelos están generalizados para buscar los mejores parámetros una vez se disponga de un conjunto de datos reales y representativos de Fletzy.
- **Entrenamiento y Validación:** Se requerirán datos históricos para entrenar estos modelos y un conjunto de pruebas separado para validar su rendimiento y comparar la efectividad de SARIMAX versus Autoregresión para este problema específico.
- **Integración:** El modelo seleccionado se integrará en el flujo de trabajo para informar la asignación de turnos y, potencialmente, para generar alertas tempranas.



Descripción de los Nodos y Flujos del Diagrama:

- **Gig Worker (vía WhatsApp):** El punto de inicio y fin de la interacción para el trabajador. Envía comandos (disponibilidad, respuesta a tareas) y recibe notificaciones.
- **API de WhatsApp Business:** Actúa como el puente de comunicación entre la aplicación WhatsApp del trabajador y el backend de la solución.
- **Servidor de Aplicaciones (Backend):** El cerebro del sistema. Orquesta las operaciones:
 - Recibe los mensajes de la API de WhatsApp.
 - Invoca el **Módulo de Gestión de Disponibilidad** para registrar o consultar horarios.
 - Invoca el **Módulo de Asignación de Tareas** cuando se deben asignar nuevos trabajos. Este módulo, a su vez, interactúa con la base de datos para conocer la disponibilidad y con el **Módulo de Modelos Predictivos** para evaluar el riesgo de ausencia.
 - Invoca el **Módulo de Notificaciones** (implícito en el flujo de retorno al trabajador).

- **Base de Datos:** Almacén central de datos (perfiles, horarios, tareas, historial para ML).
- **Módulo de Modelos Predictivos (ML):** Recibe datos del backend (historial, características del turno/trabajador) y devuelve una predicción de ausencia.
- **Administrador Fletzy / Otros Sistemas Fletzy:** Representa la posible interacción de personal de Fletzy o sistemas existentes con el backend o la base de datos para supervisión, configuración o integración de datos.

5. Estado actual de desarrollo

5.1. Funcionalidades implementadas (con capturas de pantalla)

Se implementaron 3 modelos de predicción y se hizo un cálculo de probabilidades.

5.1.1 Modelo logístico

Para el modelo logístico se usó la librería sklearn con la cual se creó y entrenó el siguiente modelo.

```
# 2. Creación y Entrenamiento del Modelo
model = Pipeline(steps=[('preprocessor', preprocessor),
                        ('classifier', LogisticRegression(solver='liblinear'))])

# Entrenar el modelo con los datos de entrenamiento
model.fit(X_train, y_train)
```

5.1.2 Cálculo de probabilidades

Para calcular las probabilidades simplemente se tomaron los datos históricos y se dividieron las faltas y asistencias por el total de días considerando la confirmación o no confirmación del gig worker.

```
def predecir_asistencia(historial, confirmacion, dia_semana):
    """
    Predice la probabilidad de asistencia basada en el historial de un individuo
    utilizando frecuencias condicionales.

    Args:
        historial (pd.DataFrame): DataFrame con el historial de asistencia,
                                   incluyendo las columnas 'Asistencia', 'Confirmación' y 'Día_Semana'.
        confirmacion (str): 'Sí' o 'No' para la confirmación futura.
        dia_semana (str): El día de la semana futuro (ej: 'Lunes').

    Returns:
        float: La probabilidad predicha de asistencia (entre 0 y 1).
        Retorna None si no hay datos históricos para esa combinación.
    """
    filtro = (historial['Confirmación'] == confirmacion) & (historial['Día_Semana'] == dia_semana)
    registros_combinacion = historial[filtro]

    if len(registros_combinacion) == 0:
        return None # No hay datos históricos para esta combinación

    asistencias = registros_combinacion['Asistencia']
    total_registros = len(asistencias)
    asistencias_si = asistencias.value_counts().get('Sí', 0)

    probabilidad_asistencia = asistencias_si / total_registros if total_registros > 0 else 0
    return probabilidad_asistencia
```

5.1.3 Modelo logístico autorregresivo

Para este modelo se hizo un modelo logístico considerando las variables $Y[t-1]$, es decir la asistencia de la semana pasada.

```
# Agregar las variables autorregresivas
data['Y_lag1'] = data['Y'].shift(1)
#data['Y_lag2'] = data['Y'].shift(2)
#data['Y_lag3'] = data['Y'].shift(3)
#data['Y_lag4'] = data['Y'].shift(4)
data = data.dropna()

# Codificar X2 (día de la semana) como variable dummy
data = pd.get_dummies(data, columns=['X2'], drop_first=True)

# Variables independientes
X_cols = ['X1', 'Y_lag1'] + [col for col in data.columns if col.startswith('X2_')]
X = data[X_cols].astype(float)
X = sm.add_constant(X)

# Variable dependiente
y = data['Y'].astype(float)

# Ajustar modelo logístico
model = sm.Logit(y, X)
result = model.fit()
```

Como se puede apreciar, es posible agregar más variables autorregresivas como $Y[t-2]$, $Y[t-3]$, etc. El criterio de agregación o no, es a través del entrenamiento y evaluación de los modelos, el cual se hará cuando se tengan los datos reales.

5.1.4 Modelo SARIMAX

Para el modelo SARIMAX, se fueron probando los diferentes parámetros del modelo SARIMAX dentro de un rango para maximizar la estacionalidad.

```
# Rango para los componentes no estacionales
p_values = range(0, 3)
q_values = range(0, 3)

# Rango para los componentes estacionales
P_values = range(0, 2)
D_values = range(0, 2)
Q_values = range(0, 2)

# Estacionalidad semanal (cada 7 días)
s = 7

mejor_aic = np.inf
mejor_orden = None
mejor_seasonal_orden = None
mejor_modelo = None

for p, q, P, D, Q in product(p_values, q_values, P_values, D_values, Q_values):
    try:
        modelo = SARIMAX(
            asistencia_diaria,
            order=(p, d, q),
            seasonal_order=(P, D, Q, s),
            enforce_stationarity=False,
            enforce_invertibility=False
        )
        modelo_fit = modelo.fit(dispatch=False)
        if modelo_fit.aic < mejor_aic:
            mejor_aic = modelo_fit.aic
            mejor_orden = (p, d, q)
            mejor_seasonal_orden = (P, D, Q, s)
            mejor_modelo = modelo_fit
    except:
        continue

print(f"Mejor orden SARIMAX encontrado: {mejor_orden} con estacionalidad {mejor_seasonal_orden}")
print(mejor_modelo.summary())
```

5.2. Integración con WhatsApp

La integración con WhatsApp todavía no se ha implementado, pero se hará para completar el flujo de funcionamiento para la asignación de turnos en tiempo real.

5.3. Resultados de pruebas preliminares

Para el modelo logístico se obtuvieron los siguientes resultados:

```
Precisión del modelo en el conjunto de prueba: 0.67

Reporte de Clasificación:
      precision    recall  f1-score   support

     0       0.00      0.00      0.00         2
     1       0.75      0.86      0.80         7

 accuracy          0.67         9
 macro avg       0.38      0.43      0.40         9
 weighted avg    0.58      0.67      0.62         9

Matriz de Confusión:
[[0 2]
 [1 6]]

Probabilidad de asistencia para el Lunes 2024-08-26: 0.47
Predicción de asistencia para el Lunes 2024-08-26: 0
Probabilidad de asistencia para el Viernes 2024-08-30: 0.53
Predicción de asistencia para el Viernes 2024-08-30: 1
Categorías One-Hot Encoder: [array(['Jueves', 'Lunes', 'Martes', 'Miércoles', 'Viernes'], dtype=object)]
Coeficientes del modelo de regresión logística: [[ 0.19545043 -0.34248844  0.27602181  0.19545043 -0.10810638]]
Intercepto del modelo de regresión logística: [0.21632785]
```

Los cuales indican la probabilidad de asistencia, la precisión del modelo y los coeficientes del modelo de regresión logística.

Para el cálculo de probabilidades, se tienen los siguientes resultados

```
Probabilidad para Monday con 'No': 0.48
Probabilidad para Wednesday con 'Sí': 0.43
Probabilidad para Friday con 'No': 0.52
Probabilidad para Saturday con 'Sí': 0.47
```

Los cuales indican la probabilidad de asistencia teniendo en cuenta la confirmación del gig worker.

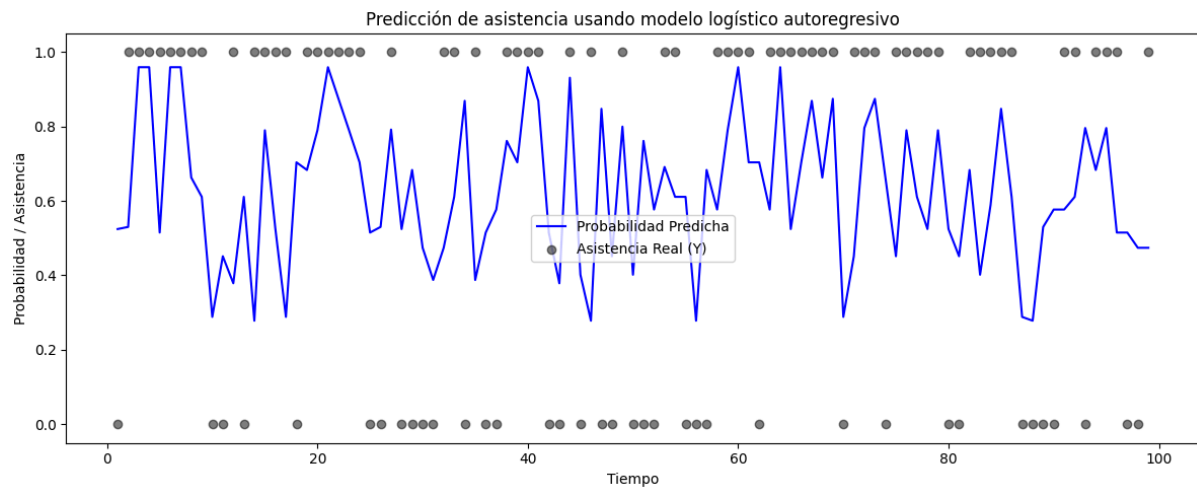
Los resultados del modelo autorregresivo son los siguientes:

```
Optimization terminated successfully.
Current function value: 0.575328
Iterations 7

Logit Regression Results
=====
Dep. Variable:                Y      No. Observations:                99
Model:                        Logit   Df Residuals:                  90
Method:                        MLE    Df Model:                      8
Date:                         Thu, 15 May 2025   Pseudo R-squ.:                0.1295
Time:                         00:19:59   Log-Likelihood:               -56.957
converged:                     True    LL-Null:                      -65.431
Covariance Type:               nonrobust   LLR p-value:                   0.03067
=====
              coef      std err          z      P>|z|      [0.025      0.975]
-----
const          1.3344      1.151      1.159      0.246     -0.922      3.591
X1             1.2626      0.501      2.522      0.012      0.281      2.244
Y_lag1         0.5548      0.475      1.167      0.243     -0.377      1.486
X2_Jueves     -2.2875      1.178     -1.942      0.052     -4.596      0.021
X2_Lunes     -1.2139      1.293     -0.938      0.348     -3.749      1.321
X2_Martes    -1.8292      1.174     -1.557      0.119     -4.131      0.473
X2_Miércoles -1.4382      1.198     -1.201      0.230     -3.785      0.909
X2_Sábado    -1.7916      1.228     -1.458      0.145     -4.199      0.616
X2_Viernes   -2.7919      1.260     -2.216      0.027     -5.261     -0.323
=====
```

Los cuales indican los coeficientes de cada una de las variables de entrada, el P valor de cada variable y la cantidad de iteraciones.

Luego, se mostró la predicción de cada asistencia y la asistencia real de cada trabajador (considerando que se usaron datos de prueba).



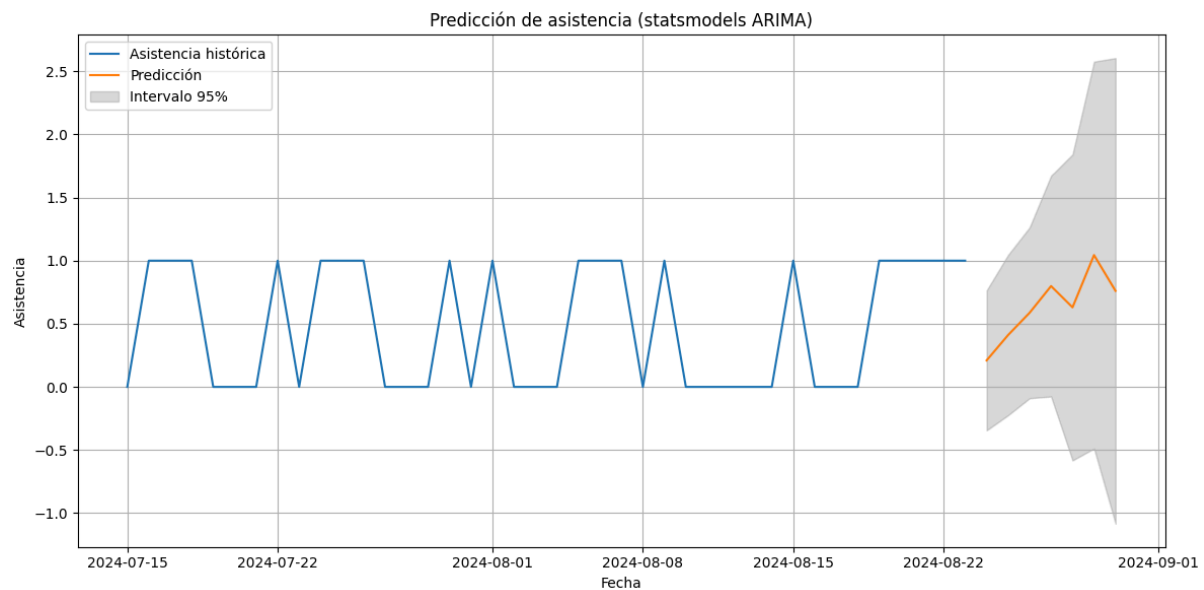
Para el modelo SARIMAX se mostró lo siguiente

SARIMAX Results						
=====						
Dep. Variable:	Asistencia		No. Observations:		40	
Model:	SARIMAX(2, 2, 2)x(1, 1, [1], 7)		Log Likelihood		-6.799	
Date:	Thu, 15 May 2025		AIC		27.597	
Time:	00:20:51		BIC		34.909	
Sample:	07-15-2024		HQIC		29.184	
	- 08-23-2024					
Covariance Type:	opg					
=====						
	coef	std err	z	P> z	[0.025	0.975]

ar.L1	0.1227	0.235	0.521	0.602	-0.338	0.584
ar.L2	-0.4720	0.149	-3.159	0.002	-0.765	-0.179
ma.L1	-1.6027	4887.817	-0.000	1.000	-9581.548	9578.343
ma.L2	1.0000	6099.573	0.000	1.000	-1.2e+04	1.2e+04
ar.S.L7	-0.7294	0.103	-7.089	0.000	-0.931	-0.528
ma.S.L7	-1.0001	5777.539	-0.000	1.000	-1.13e+04	1.13e+04
sigma2	0.0608	340.485	0.000	1.000	-667.278	667.399
=====						
Ljung-Box (L1) (Q):	0.66	Jarque-Bera (JB):	1.74			
Prob(Q):	0.42	Prob(JB):	0.42			
Heteroskedasticity (H):	0.45	Skew:	-0.67			
Prob(H) (two-sided):	0.31	Kurtosis:	2.53			
=====						

Se muestran los coeficientes y el error estándar de los parámetros del modelo SARMIAX.

Luego se mostró el siguiente gráfico



Este gráfico muestra las asistencias históricas en la línea azul y la línea naranja representa la proyección hacia el futuro, luego la región gris es el intervalo de confianza el cual se va ensanchando a medida que se va prediciendo más hacia el futuro.

5.4. Porcentaje de avance por componente

El desafío como tal se separa en 3 componentes. El primero es la **indicación de horarios de actividad**, el porcentaje de avance del grupo es del 25% ya que se tiene claro los requerimientos con los que debe cumplir la solución, por lo que falta realizar el desarrollo.

El segundo es la **predicción de faltas en base a patrones de comportamiento**, el porcentaje de avance es del 80% por ciento ya que se cuentan con 2 modelos ya generados y tan solo falta realizar pruebas con datos reales y determinar cuál es el mejor modelo en base a las capacidades y percances de cada modelo.

El tercer componente es **recibimiento de tareas en tiempo real**, el porcentaje de avance es del 25% ya que este se encuentra en el mismo estado que el primer componente donde se tienen claros los requerimientos de la solución, por lo cual tan solo resta realizar el desarrollo.

6. Desafíos y aprendizajes

6.1. Obstáculos técnicos encontrados

Obstáculos:

1. **Dudas con respecto al contexto y el desafío:**

A pesar de que en el curso tiene la posibilidad de reunirse con Fletzy los días jueves y sábados para conversar con ellos, siempre van surgiendo más dudas con respecto al contexto y al desafío designado que son difíciles de resolver en el corto tiempo de las reuniones que se tienen, y a veces estas mismas respuestas pueden causar confusión y crear más dudas que resolver.

2. **Creación y verificación de cuentas Meta Developer:**

Al momento de querer crear una cuenta de Meta Developer para integrar la API de WhatsApp, Meta le bloqueó múltiples cuentas al grupo, imposibilitando la oportunidad de realizar pruebas con respecto a la API.

3. **Falta de datos para probar el modelo:**

A pesar de tener varios modelos contruidos, para probarlos y medir distintas métricas para compararlos entre sí, se debe tener un conjunto de entrenamiento y otro de prueba para comprobar cual es mejor.

Soluciones:

1. Para el primer obstáculo se planificaron más reuniones extracurriculares con Fletzy, para tener un espacio en donde el grupo solo pueda realizarles preguntas con respecto al desafío designado y de esta manera tener más tiempo para que Fletzy responda con más detalle.
2. Se siguieron creando cuentas de Meta hasta que un miembro del grupo logró tener una cuenta la cual no bloquearon y se pueden enviar notificaciones al número de teléfono que se configuró. Por lo que dentro del grupo tenemos un integrante el cual tiene acceso a una cuenta de Meta con todas las facultades como, añadir números de teléfono, hacer uso de la aplicaciones, contar con números de prueba y la capacidad de crear mensajes de plantilla. Sin embargo, esto limita el acceso del grupo a pruebas y experimentación con el funcionamiento de cuentas comerciales de WhatsApp debido a que solo un miembro tiene acceso. Aunque actualmente nos encontramos evaluando métodos alternativos para interactuar con la API de Whatsapp

3. Debido a que no se cuentan con los datos necesarios, se generalizaron los modelos creados de manera que si se llegara a tener datos reales, estos modelos buscarán los mejores parámetros para buscar los mejores que se ajusten a los datos (se comparan creando varios modelos de la misma técnica con parámetros que tienen valores dentro de rangos asignados), y luego los modelos con los mejores parámetros de cada técnica se pueden comparar para analizar con cuál quedarse.

6.2. Adaptaciones a la propuesta inicial.

Durante el avance del proyecto, se identificó la necesidad de replantear la estructura original de la solución. Inicialmente, se consideró un sistema unificado que abordara de manera conjunta tres aspectos clave: la indicación de disponibilidad horaria, la asignación de tareas en tiempo real y la predicción de ausencias mediante modelos de machine learning. Sin embargo, a través de reuniones más frecuentes y detalladas con el equipo de Fletzy, se comprendió que estos elementos correspondían a problemáticas distintas, con objetivos y prioridades independientes.

Esta comprensión llevó a dividir el sistema en tres módulos separados, permitiendo un desarrollo más enfocado y alineado con las verdaderas necesidades de Fletzy. Por ejemplo, se nos comunicó que Fletzy ya contaba con un sistema para indicar horarios, por lo que solo requerían una adaptación puntual. En contraste, la asignación en tiempo real y el modelo de predicción eran áreas con potencial de innovación aún no explotadas por la empresa, lo que motivó una mayor inversión de tiempo y recursos en esas funcionalidades.

Además, debido a problemas técnicos con la creación de cuentas en Meta Developer y la falta de datos reales para los modelos predictivos, se optó por diseñar soluciones más flexibles y generalizables. Esto permitió avanzar en el desarrollo a pesar de los impedimentos externos, preparando el sistema para futuras integraciones una vez que se superen dichos obstáculos.

6.3. Lecciones aprendidas

Uno de los principales aprendizajes del proyecto ha sido la importancia de una comunicación constante y clara con el cliente. A pesar de tener un enunciado inicial del desafío, fue mediante reuniones regulares y sesiones de aclaración que se pudieron alinear las expectativas y redefinir el enfoque del proyecto. Esto reafirma que la interpretación temprana de los requerimientos rara vez es definitiva, y que validar constantemente las ideas con los usuarios es esencial para construir soluciones realmente útiles.

Asimismo, el equipo aprendió a adaptarse frente a las limitaciones técnicas, buscando alternativas creativas ante problemas como el bloqueo de cuentas en la plataforma de Meta o la escasez de datos. Este proceso fortaleció la capacidad de adaptación del grupo y promovió una mentalidad flexible orientada a resultados.

7. Plan de trabajo restante

7.1. Cronograma detallado de actividades pendientes

A continuación, se presentará un extracto de la carta Gantt. El documento completo se encuentra en *Documentación técnica adicional*, en la sección de Anexos.

				12/05/2025	Semana 5							Mayo			
				12/05/2025	Semana 5							Semana 6			
				12/05/2025	L	M	M	J	V	S	D	L	M	M	J
SUBTAREA				RESPONSABLE	PROGRESO	INICIO	FIN								
Actualizar Carta Gantt en relación a avances realizados				Gestor de Proyecto	Completada	14/04/2025	13/05/2025								
Realizar y actualizar informe técnico de los avances realizados				Responsable de documentación	En curso	14/04/2025	19/05/2025								
Construir presentaciones para las entregas de PEPs				Responsable de documentación	Completada	14/04/2025	13/05/2025								
Desarrollo backend para recepción de horarios				Backend Developer (API Whatsapp)	No iniciada	19/05/2025	26/05/2025								
Integración con sistema de notificaciones				Backend Developer (API Whatsapp)	No iniciada	26/05/2025	02/06/2025								
Pruebas del sistema				Backend Developer (API Whatsapp)	No iniciada	02/06/2025	04/06/2025								
Análisis de requisitos del sistema de asignación				Responsable de Asignación de Turnos	No iniciada	19/05/2025	21/05/2025								
Diseño de algoritmos de asignación de turnos				Responsable de Asignación de Turnos	No iniciada	21/05/2025	04/06/2025								
Desarrollo de la lógica de asignación de turnos en tiempo real				Responsable de Asignación de Turnos	No iniciada	04/06/2025	11/06/2025								
Integración con base de datos y sistema de disponibilidad				Responsable de Asignación de Turnos	No iniciada	12/06/2025	16/06/2025								
Pruebas del sistema de asignación				Responsable de Asignación de Turnos	No iniciada	16/06/2025	18/06/2025								
Formateo de datos				Desarrollador de Modelos Predictivos	No iniciada	19/05/2025	02/06/2025								
Entrenamiento y validación del modelo				Desarrollador de Modelos Predictivos	No iniciada	02/06/2025	23/06/2025								
Definición de flujo de notificación				Backend Developer (API Whatsapp)	No iniciada	04/06/2025	06/06/2025								
Desarrollo de API para enviar confirmaciones				Backend Developer (API Whatsapp)	No iniciada	09/06/2025	13/06/2025								
Integración con el sistema de confirmaciones				Backend Developer (API Whatsapp)	No iniciada	16/06/2025	20/06/2025								
Pruebas del sistema de envío				Backend Developer (API Whatsapp)	No iniciada	23/06/2025	30/06/2025								
Todos				En curso		17/04/2025	10/07/2025								

7.2. Distribución de tareas entre integrantes

Para la distribución de actividades de la Carta Gantt, se determinaron distintos roles que se asocian a actividades relacionadas a la descripción del rol. A continuación, se presentan los roles determinados, su descripción y los integrantes de cada uno.

Rol	Descripción de responsabilidades y funciones asociadas al rol	Integrante
Gestor de Proyecto	Se encarga de actualizar la planificación de actividades.	Stephan Paul
Backend Developer (Integración con API Whatsapp)	Se encarga del desarrollo e integración de la API de Whatsapp.	- Benjamín Moya - Víctor Duarte
Responsable de documentación	Se encarga de realizar informes y presentaciones de avances.	Alonso Henriquez
Desarrollador de Modelos Predictivos	Se encarga de desarrollar el modelo para predecir asistencia.	- Víctor Duarte - Stephan Paul
Responsable de Asignación Dinámica de Turnos	Se encarga de la optimización de turnos.	- Alonso Henriquez - Milovan Valenzuela

7.3. Estimación de esfuerzo requerido

A continuación, se presenta una tabla con la asignación de tiempos y esfuerzos por actividad de la Carta Gantt.

Actividad/Subtarea	Responsable	Horas por responsable (HH)
Gestión del proyecto	-	-
Actualizar Carta Gantt en relación a avances realizados	Gestor de Proyecto	3
Realizar y actualizar informe técnico de los avances realizados	Responsable de documentación	5
Construir presentaciones para las entregas de PEPs	Responsable de documentación	4
Desafío 1: Confirmación de los gig workers	-	-
Desarrollo backend para recepción de horarios	Backend Developer (API Whatsapp)	6
Integración con sistema de notificaciones	Backend Developer (API Whatsapp)	6
Pruebas del sistema	Backend Developer (API Whatsapp)	4
Desafío 2: Asignación de turnos en tiempo real	-	-
Análisis de requisitos del sistema de asignación	Responsable de Asignación de Turnos	3
Diseño de algoritmos de asignación de turnos	Responsable de Asignación de Turnos	6
Desarrollo de la lógica de asignación de turnos en tiempo real	Responsable de Asignación de Turnos	8
Integración con base de datos y sistema de disponibilidad	Responsable de Asignación de Turnos	5
Pruebas del sistema de asignación	Responsable de Asignación de Turnos	4
Desafío 3: Modelo de predicción (ML)	-	-
Formateo de datos	Desarrollador de Modelos Predictivos	5

Entrenamiento y validación del modelo	Desarrollador de Modelos Predictivos	8
Desafío Extra: Notificación a número empresarial	-	-
Definición de flujo de notificación	Backend Developer (API Whatsapp)	3
Desarrollo de API para enviar confirmaciones	Backend Developer (API Whatsapp)	6
Integración con el sistema de confirmaciones	Backend Developer (API Whatsapp)	5
Pruebas del sistema de envío	Backend Developer (API Whatsapp)	4

8. Anexos

8.1. Código relevante (extractos):

- Acceso directo a código (modelos de predicción):
https://colab.research.google.com/drive/1HMPXH7IVFXTpf-Q50IAZLa5BM_vpT56R?usp=sharing

SARIMAX

```
# Convertir a serie temporal diaria
asistencia_diaria = df.groupby('Fecha')['Asistencia'].sum()
asistencia_diaria = asistencia_diaria.asfreq('D').fillna(0).astype(int)

# -----
# Paso 1: Estimar automáticamente d con ADF
# -----
def estimar_d(series, max_d=2):
    for d in range(max_d + 1):
        serie_diff = series.diff(d).dropna()
        # Verifica si la serie es constante
        if serie_diff.nunique() <= 1:
            continue
        try:
            p_value = adfuller(serie_diff)[1]
            if p_value < 0.05:
                return d
        except:
            continue
    return max_d

d = estimar_d(asistencia_diaria)
```

Autoregresión

```
# Crear variable Y con dependencia de X1 y Y[t-1]
data['Y'] = 0
data.loc[0, 'Y'] = np.random.binomial(1, 0.5)

for i in range(2, n):
    p = 1 / (1 + np.exp(-(-0.5 + 1 * data.loc[i, 'X1'] + 0.2 * data.loc[i-1, 'Y'])))
    data.loc[i, 'Y'] = np.random.binomial(1, p)

# Agregar las variables autorregresivas
data['Y_lag1'] = data['Y'].shift(1)
#data['Y_lag2'] = data['Y'].shift(2)
#data['Y_lag3'] = data['Y'].shift(3)
#data['Y_lag4'] = data['Y'].shift(4)
data = data.dropna()

# Codificar X2 (día de la semana) como variable dummy
data = pd.get_dummies(data, columns=['X2'], drop_first=True)

# Variables independientes
X_cols = ['X1', 'Y_lag1'] + [col for col in data.columns if col.startswith('X2_')]
X = data[X_cols].astype(float)
X = sm.add_constant(X)

# Variable dependiente
y = data['Y'].astype(float)

# Ajustar modelo logístico
model = sm.Logit(y, X)
result = model.fit()
```

8.2. Documentación técnica adicional

- Carta Gantt:  Carta Gantt IE.xlsx